Yes — for this project, you can get data in **3 practical ways**:

1. Simulated Data — Best for MVP

Since you do not have real routers/switches, create a **Python device simulator**.

It generates fake but realistic network telemetry:

```
{
  "device_id": "router-01",
  "cpu": 82,
  "memory": 70,
  "latency_ms": 180,
  "packet_loss": 3.2,
  "bandwidth_mbps": 650,
  "status": "warning"
}
```

You simulate:

- normal device behavior
- CPU spikes
- packet loss
- latency increase
- device offline
- config drift
- recovery workflow

This is enough to prove the full backend.

---

2. Public / Sample Network Data

Use sample datasets for anomaly detection training/testing:

- network intrusion datasets
- server monitoring datasets
- telecom anomaly datasets
- synthetic time-series datasets

But for your project, **simulated real-time data is better** because the role is about building/testing/automation workflows.

---

3. Real Local Machine Metrics

You can collect real data from your own laptop using Python:

```
import psutil

cpu = psutil.cpu_percent()
memory = psutil.virtual_memory().percent
network = psutil.net_io_counters()
```

Use your laptop as one “device,” then simulate 50 more devices.

Best Data Plan

Use a hybrid approach:

```
Real laptop metrics
      +
Simulated routers/switches
      +
Injected failure scenarios
```

That makes the project look realistic.

How to Test the Workflow

You test each feature by forcing controlled scenarios.

Workflow 1: Normal Monitoring

Input:

CPU: 40%
Latency: 50ms
Packet loss: 0.2%

Expected result:

Device status = Healthy
No incident created
No alert triggered

Workflow 2: High CPU Alert

Input:

CPU: 95%

Expected result:

Alert created
Device marked Warning
Dashboard updates through WebSocket
Incident created if CPU stays high

Workflow 3: Device Offline

Input:

No heartbeat for 30 seconds

Expected result:

Device status = Offline
Critical alert created
Automation workflow triggered
Incident opened

Workflow 4: Config Drift

Expected config:

vlan: 20
firewall_enabled: true

Actual config:

vlan: 30
firewall_enabled: false

Expected result:

Drift detected
Compliance status = Failed
Rollback recommended
Automation triggered

Workflow 5: Auto-Recovery

Scenario:

Device offline

Automation should:

1. Create incident
2. Try restart simulation
3. Recheck heartbeat
4. Mark recovered or unresolved
5. Save automation log

Testing Types You Should Add

Test Type	What It Validates
Unit tests	health score, anomaly rules, config drift
API tests	FastAPI endpoints
Integration tests	simulator → stream → DB → alert
WebSocket tests	real-time dashboard updates
Load tests	100+ simulated devices
Failure tests	offline device, bad config, broken worker

Best Testing Tools

pytest
pytest-asyncio
httpx
Locust
Docker Compose
Postman
Prometheus/Grafana

Complete Test Flow

Device Simulator
↓
Send telemetry
↓
Redis Stream receives data
↓
Telemetry worker processes event
↓
Rules engine checks thresholds
↓
AI engine checks anomaly
↓
Alert/incident created

↓

Automation workflow runs

↓

Database stores result

↓

Dashboard updates live

Best Data Type for This Project

You should collect/simulate these 5 categories:

Data Category	Example Fields	Why It Matters
Device inventory	device_id, type, location, IP, firmware	Knows what devices exist
Telemetry metrics	CPU, memory, latency, packet loss, bandwidth	Core monitoring data
Heartbeat data	device_id, timestamp, status	Detects online/offline devices
Configuration data	VLAN, firewall, routing protocol, firmware version	Supports config drift + rollback
Event/alert data	severity, reason, action_taken, timestamp	Tracks incidents and automation

Best Real-Time Event Format

Send one telemetry event every **2–5 seconds per device**:

{

"event_id": "evt-1001",

"device_id": "router-01",

"device_type": "router",

```
  "location": "San Jose DC-1",
  "timestamp": "2026-05-17T10:30:00Z",
  "cpu_percent": 82.5,
  "memory_percent": 71.3,
  "latency_ms": 145,
  "packet_loss_percent": 2.1,
  "bandwidth_mbps": 640,
  "interface_status": "up",
  "health_status": "warning"
}
```


Best Practice Real-Time Pipeline

Device Simulator / Local Agent

↓

Redis Streams or Kafka

↓

Telemetry Consumer Worker

↓

Rules Engine + AI Anomaly Detection

↓

TimescaleDB/PostgreSQL

↓

WebSocket API

↓

React Dashboard

Best Practice for Realistic Simulation

Create **normal + failure scenarios**:

Normal Data

CPU: 20–60%

Memory: 30–70%

Latency: 20–80 ms

Packet loss: 0–1%

Warning Scenario

CPU: 75–90%

Latency: 120–200 ms

Packet loss: 2–5%

Critical Scenario

CPU: 90–100%

Latency: 250+ ms

Packet loss: 5%+

No heartbeat for 30 seconds

Best Data Source Strategy

Use this combination:

80% simulated router/switch/network telemetry

+

20% real laptop/server metrics using psutil

That makes the project feel realistic and testable without hardware.

Best Practice Database Split

Data	Store In
Device inventory	PostgreSQL
Config versions	PostgreSQL
Incidents/alerts	PostgreSQL
Time-series telemetry	TimescaleDB
Live status/cache	Redis

Most Important Realtime Feature

The dashboard should update instantly when:

CPU spike happens

Device goes offline

Packet loss increases

Config drift is detected

Automation recovery runs

Incident is resolved

That proves the project is truly real-time.